

Task Day08

How can we handle security issues concerning connection string written as hardcode name 3 approaches to do this ??

Hardcoding a connection string in code (or even in AppDbContextFactory) is a security and maintenance risk. In .NET, there are several better approaches to handle it safely.

Here are **3 common secure approaches**:

1) appsettings.json (with environment separation)

Store the connection string in appsettings.json instead of code:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=.;Database=MyDb;Trusted_Connection=True;"
  }
}
```

Then read it in Program.cs:

```
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
```

✓ Improvement:

Use environment-specific files:

- appsettings.Development.json
- appsettings.Production.json

So production credentials are not exposed in development.

2) User Secrets (Development only)

Best for local development (keeps secrets out of source code).

Initialize:

dotnet user-secrets init

Add connection string:

```
dotnet user-secrets set "ConnectionStrings:DefaultConnection"  
"Server=.;Database=MyDb;Trusted_Connection=True;"
```

.NET automatically reads it in Development mode.

✓ Advantages:

- Not stored in project files
 - Not pushed to GitHub
 - Very safe for local dev
-

3) Environment Variables (Production ready)

Store connection string in OS / server environment variables.

Example:

```
ConnectionStrings__DefaultConnection=Server=.;Database=MyDb;Trusted_Connection=Tru  
e;
```

.NET automatically maps:

So it becomes:

```
Configuration.GetConnectionString("DefaultConnection")
```

Advantages:

- Perfect for cloud / Docker / CI-CD
- No secrets in code or files
- Easy to rotate credentials

What is AJAX , mention 3 reasons to use in MVC apps?? Implement in simple demo

AJAX (Asynchronous JavaScript and XML) is a technique used in web applications to send and receive data from the server without reloading the whole page.

In modern apps (like ASP.NET MVC), AJAX is usually used with JSON instead of XML.

3 Reasons to use AJAX in MVC apps

1. No full page reload
 - Only part of the page updates → better user experience
 2. Faster performance
 - Less data transferred and faster UI updates
 3. Better interactivity
 - Enables real-time features like search, filtering, comments, etc.
-

Simple AJAX Demo in ASP.NET MVC

We will create a simple example:

Click button → load products without refreshing page

1) Model

```
public class Product
{
    public int Id { get; set; }
    public string? Name { get; set; }
}
```

2) Controller

```
using Microsoft.AspNetCore.Mvc;
```

```
public class ProductController : Controller
{
    public IActionResult Index()
    {
        return View();
    }
}
```

```
public JsonResult GetProducts()
{
    var products = new List<Product>
    {
        new Product { Id = 1, Name = "Laptop" },
        new Product { Id = 2, Name = "Mouse" },
        new Product { Id = 3, Name = "Keyboard" }
    };

    return Json(products);
}
}
```

3) View (Index.cshtml)

```
<button onclick="loadProducts()">Load Products</button>
```

```
<ul id="productList"></ul>
```

```
<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
```

```
<script>
function loadProducts() {
    $.ajax({
        url: '/Product/GetProducts',
        type: 'GET',
        success: function (data) {
            let html = "";

            data.forEach(p => {
                html += `<li>${p.name}</li>`;
            });

            $('#productList').html(html);
        },
        error: function () {
            alert("Error loading data");
        }
    });
}
</script>
```
